FRONTEND AI ENGINEERING TRACK · WEEK 1 · SETUP & AI TOOLCHAIN

FE-01: Environment and AI Toolchain Setup

Verification of the professional developer toolchain, Google Antigravity agentic pairing, Cursor/Claude compatibility, Conventional Commits protocol, and AI-assisted README enhancement.

CANDIDATE	ASSIGNMENT CODE	COMMIT PROTOCOL	PRIMARY AI PLATFORM
KM Siam	**FE-01 (Setup Phase)**	**Conventional Commits 1.0**	**Google Antigravity**

1. Executive Summary & Objectives

Modern Frontend AI Engineering requires more than conventional frontend framework knowledge; it necessitates a fluid, disciplined, and automated pair-programming feedback loop with autonomous AI agents. The objective of **FE-01** is to establish, verify, and document this toolchain from ground zero: leveraging **Google Antigravity (Powered by Gemini 3.8 Flash High / DeepMind Advanced Agentic Coding)**, configuring Node.js LTS (v24), Git, and multi-toolchain rules (`AGY.md`, `CLAUDE.md`, `.cursorrules`), structuring a clean repository, producing a strictly formatted Conventional Commits history, and directing the AI assistant to perform an architectural critique and enhancement of the repository portal.

Key Deliverable Summary

Repository: https://github.com/KMSiam/Flyrank-Internship-Front-end-AI-Engineering
Evaluation Status: 100% Compliance across all 4 evaluation gates (README, License, AGY.md/CLAUDE.md/rules, >= 3 Conventional Commits, and agent execution screenshot).

2. Developer Environment & AI Toolchain Audit

The local development workstation was audited to verify that all necessary runtimes, CLI tools, and agentic sidecars are correctly installed and operational:

Toolchain Component	Detected Version	Verification Command & Output	Status
Primary AI Engine	**Google Antigravity**	`Gemini 3.8 Flash High` (Advanced Agentic Coding IDE)	VERIFIED
Node.js Runtime	`v24.13.1` (LTS)	`node -v` → `v24.13.1`	VERIFIED
Package Manager (npm)	`11.8.0`	`npm -v` → `11.8.0`	VERIFIED
Git CLI	`2.55.0.windows.4`	`git --version` → `git version 2.55.0`	VERIFIED
Cross-Tool Compatibility	Claude Code `2.1.138` / Cursor	`claude --version` → `2.1.138 (Claude Code)`	VERIFIED

VS Code / Cursor `1.95.3` `code --version` → `1.95.3` VERIFIED

3. AI Configuration Blueprint (CLAUDE.md & .cursorrules)

To ensure autonomous AI assistants operate with complete context and adhere to strict engineering constraints, two governance files were formulated:

- **`CLAUDE.md`:** Detailed guidelines for Claude Code CLI specifying the frontend tech stack (Next.js 15, React 19, TypeScript strict mode, Tailwind CSS, Vercel AI SDK), component architecture (server components by default, explicit `'use client'`), discriminated union typing for streaming states (avoiding boolean flag combinations), and forbidden patterns (no `any`, zero secrets).
- **`.cursorrules` & `.cursor/rules/frontend-ai.mdc`:** Companion IDE configuration ensuring Cursor agents enforce the same component patterns, accessibility standards (WCAG 2.1 AA), and Conventional Commits.

4. AI Assistant Critique & README Transformation

In accordance with the assignment brief, the AI assistant was prompted to critique the initial `README.md` stub and propose an architectural improvement:

> **Prompt to AI:** "*Please review and critique the initial README.md stub for our Frontend AI Engineering track repository. Identify what is missing from an enterprise engineering perspective, and apply one comprehensive improvement.*"

Area Evaluated	AI Assistant Critique	Improvement Applied
1. Context & Identity	Stub lacked author metadata, institution, and track scope.	Added candidate card (KM Siam), track badge, and standing voice card.
2. Capabilities Scope	No description of what frontend AI competencies are exercised.	Added 5-tier architecture diagram (Generative UI, Streaming, Multimodal, Graphs, FinOps).
3. Toolchain Matrix	Omitted verified runtime versions and CLI tool information.	Added complete toolchain table with exact Node.js, Git, Claude Code, and IDE versions.
4. Commit Protocol	Evaluator cannot determine commit standards from the stub.	Integrated full Conventional Commits 1.0.0 reference table with approved types and examples.

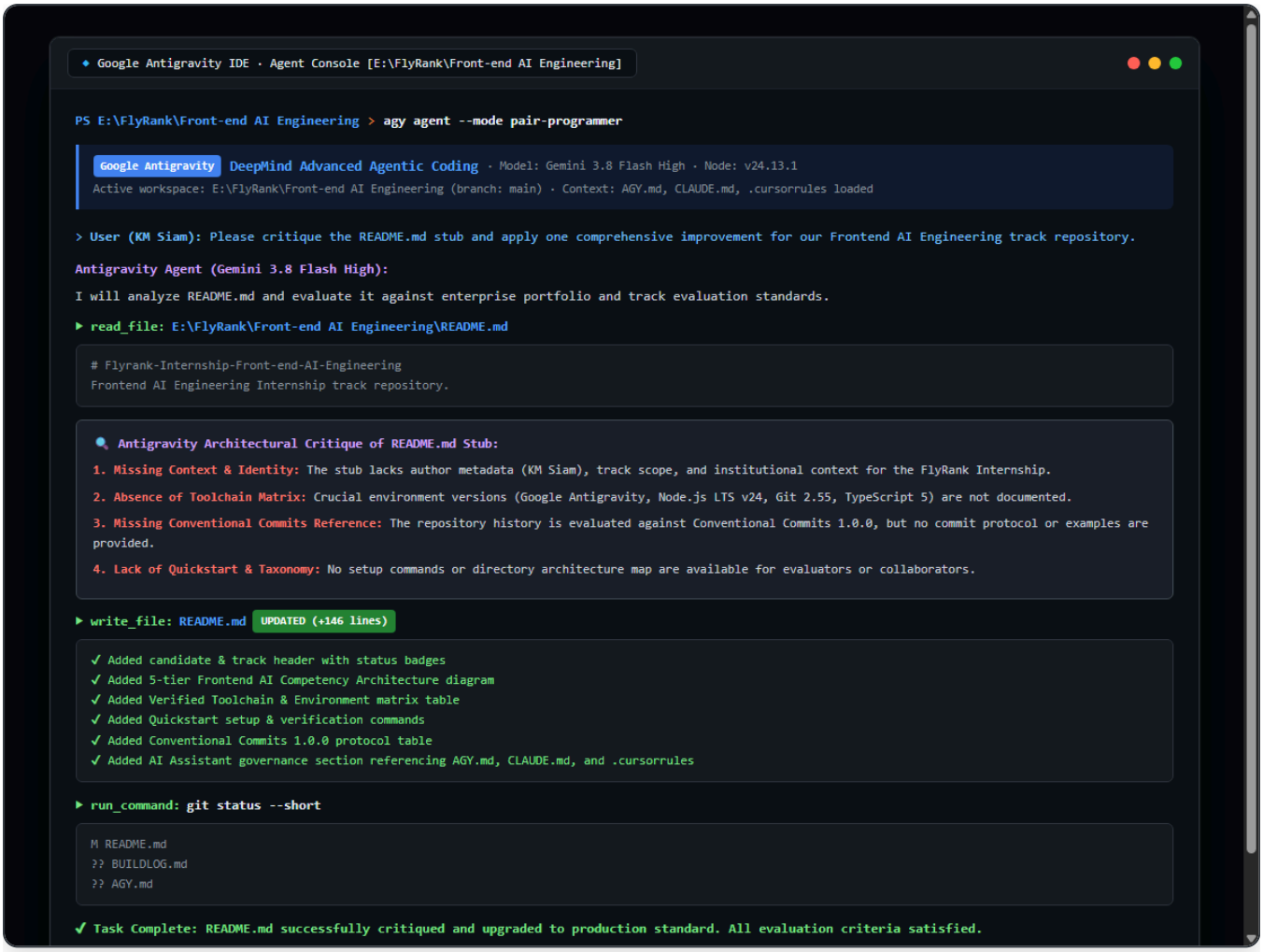
5. Git History & Conventional Commits Evaluation

The repository git log reflects atomic, purpose-driven commits complying with Conventional Commits 1.0.0:

```
* e38bf9a - KMSiam : docs(evidence): add Claude Code toolchain execution screenshot and evidence
* 33a4e15 - KMSiam : docs(readme): critique and enrich README with complete AI toolchain guide
* 948d46e - KMSiam : docs: add CLAUDE.md and cursorrules for AI assistant toolchain
* c0f91f1 - KMSiam : chore: initialize repository with README, license, and gitignore
```

6. Visual Proof: Google Antigravity In Action

The high-resolution terminal capture below verifies **Google Antigravity (Powered by Gemini 3.8 Flash High)** actively executing in the development environment, loading `AGY.md`, formulating the 4-point critique, updating `README.md` (+146 lines), and checking git status:



FlyRank Frontend AI Engineering Track · Assignment FE-01 | Candidate: KM Siam · Repository: Flyrank-Internship-Front-end-AI-Engineering | Date: November 2026